P2 ESP32: WiFi, GPIO y programación con Arduino IDE

Semana	19–23 oct 2026
Imparte	IES Politècnic
Duración	2 sesiones de 50 min + práctica en el aula
Material	Placa ESP32 DevKit (ESP32-WROOM-32), cable USB de datos, DHT11, LED + 220 Ω, datos de la WiFi del aula (red IoT si existe)
Requisitos previos	P1

Objetivos

- Preparar Arduino IDE para programar el ESP32.
- Conocer las diferencias eléctricas entre ESP32 y Arduino UNO (3,3 V).
- Conectar el ESP32 a la WiFi del aula y comprobar su IP.
- Publicar las lecturas del DHT11 en una pequeña página web servida por el propio ESP32.

Contenidos

1. Qué es el ESP32

El ESP32 es un microcontrolador de doble núcleo a 240 MHz con **WiFi y Bluetooth** integrados. En domótica lo usamos como nodo conectado: lee sensores, gobierna actuadores y se comunica con Home Assistant por la red.

Característica	Arduino UNO	ESP32 DevKit
Tensión lógica	5 V	3,3 V (¡no tolera 5 V en sus pines!)
Conectividad	Ninguna	WiFi 2,4 GHz y Bluetooth
ADC	10 bits (0–1023)	12 bits (0–4095)
Puertos serie	1 (USB)	3 (UART0 por USB, UART1, UART2)
Memoria de programa	32 KB	4 MB de flash

> Pines a evitar en el ESP32 DevKit: GPIO 6–11 (memoria flash interna), GPIO 34–39 solo son entradas y no tienen pull-up, y los pines del ADC2 (p. ej. 0, 2, 4, 12–15, 25–27) no leen analógico con la WiFi activa: usa el ADC1 (GPIO 32–39). Para entradas analógicas usaremos GPIO 34 o 35; para salidas digitales, GPIO 4, 5, 16–19, 21–23, 25–27, 32 o 33.

2. Preparar Arduino IDE

- Archivo → Preferencias → «URLs adicionales de gestor de placas»: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
- Herramientas → Placa → Gestor de placas → busca «esp32» (de Espressif Systems) e instálalo.
- Selecciona la placa «ESP32 Dev Module» y el puerto. Si la carga no arranca, mantén pulsado BOOT hasta que empiece a escribir.

Proyecto de agrupación de centros financiado por el Ministerio de Educación, Formación Profesional y Deportes.
El contenido de esta publicación compromete exclusivamente a sus autores.

Sketch 2 · WiFi + servidor web con temperatura y humedad

```
#include <WiFi.h>
#include <WebServer.h>
#include <DHT.h>

const char* SSID = "RED_IOT_AULA";       // pon aquí la red de tu aula
const char* PASS = "********";           // no subas la contraseña real a GitHub

DHT dht(4, DHT11);                       // DHT11 en GPIO4, alimentado a 3,3 V
WebServer server(80);

void paginaInicio() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();
  String html = "<h1>Nodo clima</h1>";
  html += "<p>Temperatura: " + String(t, 1) + " &deg;C</p>";
  html += "<p>Humedad: " + String(h, 1) + " %</p>";
  server.send(200, "text/html; charset=utf-8", html);
}

void setup() {
  Serial.begin(115200);
  dht.begin();
  WiFi.mode(WIFI_STA);
  WiFi.begin(SSID, PASS);
  Serial.print("Conectando");
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.print("\nConectado. IP: ");
  Serial.println(WiFi.localIP());
  server.on("/", paginaInicio);
  server.begin();
}
void loop() {
  server.handleClient();
}
```

3. Buenas prácticas de red

- El ESP32 solo funciona en **2,4 GHz**: una red solo de 5 GHz no aparecerá.
- En el aula conviene una red o VLAN separada para los dispositivos IoT (lo veremos en P10).
- Pide al profesor que reserve una IP fija para cada nodo en el router (reserva DHCP) para localizarlo siempre.

Práctica simultánea en el aula

1. Añade la URL de placas ESP32, instálalas y carga el ejemplo *WiFi → WiFiScan*: comprueba que ves la red del aula.
2. Conecta el DHT11 al ESP32 alimentado a **3,3 V** con el dato en GPIO4 y el LED en GPIO5 con su resistencia.
3. Carga el Sketch 2 con los datos de la red y anota la IP que aparece en el monitor serie (115200 baudios).
4. Desde el móvil o el PC, en la misma red, abre *http://<IP>*: debes ver la página del nodo.
5. Añade una segunda ruta */led/on* y */led/off* con server.on(...) que encienda y apague el LED.
6. Sube el código a tu carpeta de practicas-pildoras con una pull request, sustituyendo la contraseña por asteriscos.

Proyecto de agrupación de centros financiado por el Ministerio de Educación, Formación Profesional y Deportes.
El contenido de esta publicación compromete exclusivamente a sus autores.

Lista de comprobación (evidencias)

- ☐ El ESP32 se conecta a la WiFi y muestra su IP.
- ☐ La página web muestra temperatura y humedad actualizadas al recargar.
- ☐ El LED se controla desde el navegador.
- ☐ Código subido sin contraseñas reales.

Errores frecuentes

Síntoma	Solución
«Failed to connect to ESP32: Timed out waiting for packet header»	Mantén pulsado BOOT al empezar la carga o baja la velocidad de subida a 115200.
Se queda en «Conectando.....» para siempre	Comprueba SSID y contraseña (mayúsculas) y que la red es de 2,4 GHz.
Lecturas nan del DHT11	Revisa que el sensor está a 3,3 V y el pin de datos es GPIO4 como en el código.
El ESP32 se reinicia al conectar la WiFi	Alimentación insuficiente: usa otro puerto USB o un cable más corto.

Para ir más allá (nivel avanzado)

- Haz que la página se recargue sola cada 10 s añadiendo <meta http-equiv='refresh' content='10'>.
- Lee la intensidad de señal con WiFi.RSSI() y muéstrala en la página: ¿dónde tiene mejor cobertura el aula?

Conexión con FpAR y autoevaluación

> **Competencia transversal: Alfabetización tecnológica** (resolución de problemas técnicos y seguridad digital: no publicar credenciales).

Tres preguntas para comprobar lo aprendido:

1. ¿Por qué no podemos conectar directamente una salida de 5 V a un pin del ESP32?
2. ¿Qué pines analógicos del ESP32 siguen funcionando con la WiFi activa?
3. ¿Para qué sirve reservar una IP fija a cada nodo?

Proyecto de agrupación de centros financiado por el Ministerio de Educación, Formación Profesional y Deportes.
El contenido de esta publicación compromete exclusivamente a sus autores.